Week 2 Handout

Untyped and Typed Lambda Calculus, Dependent Types, and Inductive Mathematics

Course: AI for Mathematics
Instructor: Ma Jiajun
Audience: XMUM General Elective

---

0. Guiding Question

This lecture is organized around one question:

> What is a type, and why does type theory treat computation and proof inside one language?

Week 1 let us use Lean as a tool. Week 2 explains the ideas behind that tool.

The lecture has three goals:

1. Start from untyped lambda calculus as a pure model of computation.
2. Explain why types must be introduced as discipline.
3. Build from simple types to dependent types and inductive mathematics.

---

1. The Story of Untyped Lambda Calculus

We begin with the strongest conceptual surprise:

> computation can be described before types are added.

This gives students a clean mental model:

- first, there is a bare language of functions and substitution;
- then, there is the question of how to control that language;
- only after that do types enter as discipline.

Historical Role of Alonzo Church

```
<strong>Alonzo Church</strong> (1903–1995)
introduced lambda calculus as a formal account
of function definition and application.
His work is one of the conceptual foundations
of functional programming and proof
assistants.
```

Students should leave this opening section with the idea that:

- untyped lambda calculus is minimal,
- yet still computationally universal,
- and already supports a surprising amount of structure before any type discipline is added.

Core Story

The untyped story should be told in this order:

1. computation can be modeled using only variables, abstraction, and application;
2. function application is substitution;
3. data can be represented by behavior;
4. the system is powerful, but too permissive;
5. that is why typed lambda calculus becomes necessary.

Syntax of Lambda Calculus

There are three basic term forms:

1. variable
2. abstraction
3. application

In notation:

- `x`
- `λx. e`
- `(f a)`

This should be emphasized carefully:

- a variable by itself does not compute,
- an abstraction is a function waiting for input,
- an application is the place where computation can actually happen.

Beta Reduction

The central computation rule is:

```
(λx. e) v  →  e[v/x]
```

This means: apply a function by substitution.

Examples:

```
(λx. x) 5  →  5

(λx. x + 1) 4  →  5

(λx. λy. x) 3 7  →  (λy. 3) 7  →  3
```

A Classroom Example

For Week 2, one good worked example is:

```
(λf. λx. f (f x)) (λz. z + 1) 3
```

This reduces in stages:

```
(λx. (λz. z + 1) ((λz. z + 1) x)) 3
→ (λz. z + 1) ((λz. z + 1) 3)
→ (λz. z + 1) 4
→ 5
```

This example shows:

- nested abstraction,
- repeated function application,
- computation as repeated substitution.

It is also a good example because students can still follow every line by hand. That matters: at this stage, lambda calculus should feel executable on paper.

Alpha Equivalence

Bound variable names do not matter:

```
λx. x  ≡α  λy. y
```

Students should understand that meaning depends on binding structure, not on the literal choice of variable name.

The important distinction is between:

- a bound variable, whose name can be changed safely,
- and a free variable, whose surrounding context matters.

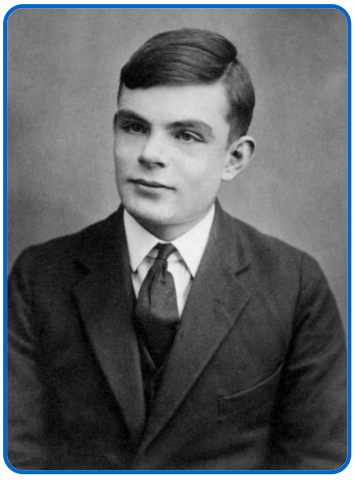
Lean Anchor: Binding Structure Matters More Than Names

```
variable (y : Nat)

def f := fun x => x^2 + 1
def g := fun y => y^2 + 1

variable (x y : Nat)

abbrev F := fun z : Nat => z^2 + y

#check F x

example : F x = fun z => z^2 + x := rfl
example : F x y = y^2 + x := rfl
-- example : F x y = y^2 + y := rfl -- wrong
```

This is the right place to explain:

- `f` and `g` have the same meaning despite different bound variable names,
- but `F` depends on a free variable from the surrounding context,
- so changing the surrounding variable changes the term.

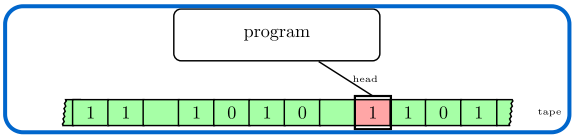
Historical Note: Church and Turing



```
<strong>Alan Turing</strong> (1912–1954)
developed the Turing machine model in the same
foundational period.
The Church–Turing thesis expresses the idea
that lambda calculus and Turing machines
capture the same general notion of effective
computation.
```

Turing Machine: The Other Foundational Model

To complement lambda calculus, the lecture should briefly introduce the Turing machine.




```
A <strong>Turing machine</strong>
is an abstract computing device
with:
<ul>
  <li>a tape divided into cells,
</li>
  <li>a head that reads and
writes symbols,</li>
```

```
  <li>a finite list of states,
</li>
  <li>and rules saying what to do
next.</li>
</ul>
It gives a very different-looking
model of computation from lambda
calculus:
one is based on symbolic
rewriting of functions, the other
on state transitions over a tape.
```

How a Turing Machine Runs

At each step, the machine does exactly one local action:

1. read the symbol currently under the head,
2. consult the current state,
3. use the transition rule for that state-symbol pair,
4. write a symbol,
5. move left or right,
6. change to a new state.

So a computation is a long sequence of very small, explicit steps.

Why This Model Is Good for Algorithms

A Turing machine is useful because it separates three things very clearly:

- **memory**: the tape,
- **control**: the finite state table,
- **execution**: repeated local transitions.

That is why it became a canonical model of algorithmic procedure.

Why Mention It Here

Students should hear the comparison explicitly:

- **lambda calculus** models computation as substitution in symbolic expressions;
- **Turing machines** model computation as step-by-step state evolution on memory;
- the Church–Turing thesis says these radically different models capture the same class of effective procedures.

This also explains why computability theory has both:

- a syntactic/function-theoretic model,
- and a machine/process model.

Hilbert, the Decision Problem, and the Halting Problem

```
<strong>David Hilbert</strong> (1862–1943)
proposed a broad program for making
mathematics precise, complete, and
mechanically tractable.
In this setting, one natural question became
the <strong>Entscheidungsproblem</strong>: is
there a general procedure that can decide
whether a formal mathematical statement is
provable?
```

This is the right historical place to mention two closely related ideas:

- the **Entscheidungsproblem**, associated with Hilbert's program,
- the **halting problem**, formulated in machine terms by Turing.

The precise point for students is:

- the halting problem is not one of Hilbert's original 23 problems,
- but it is part of the same foundational period and gives a machine-theoretic route to a negative answer about universal decision procedures.

The Halting Problem

The halting problem asks:

> Is there a single algorithm that can always decide whether an arbitrary program will eventually stop?

Turing's answer is no.

This matters in Week 2 because it shows that:

- even extremely simple-looking questions about computation can be undecidable,
- and the study of formal systems naturally leads to limits of computation,
- which is why Church and Turing are not just inventors of models, but discoverers of impossibility results.

This gives the lecture a stronger historical arc:

- formal systems are introduced to clarify mathematics,
- formal systems also reveal what mathematics and computation cannot do mechanically.

---

2. A First Surprise: Data as Behavior

This section should stay intuitive. The point is not to make students manipulate full encodings in detail. The point is to show that in untyped lambda calculus, familiar data can be represented by what it does.

Booleans as Choice

The simplest example is a boolean.

Instead of thinking of `true` and `false` as tiny stored objects, we can think of them as ways of choosing between two alternatives.

In the Church encoding:

```
true  = λt. λf. t
false = λt. λf. f
```

So if we give such a term two options, it chooses one of them:

```
if b then e1 else e2  =  b e1 e2
```

This is the main intuition:

- `true` means "pick the first branch,"
- `false` means "pick the second branch."

At this stage, students do not need more than that. The key point is that a boolean can be understood operationally as a selector.

Lean Anchor: Church Boolean as a Selector

```
abbrev CBool := {α : Type} → α → α → α

def ctrue : CBool := fun {_} t f => t
def cfalse : CBool := fun {_} t f => f

def cIte (b : CBool) (t e : α) : α :=
  b t e

#eval cIte ctrue 10 20
#eval cIte cfalse 10 20
```

This code belongs here because it matches the exact idea of the section:

- a boolean is not treated as stored data,
- it is treated as a choice operator.

Church Numerals

Natural numbers can also be encoded functionally:

```
0 = λf. λx. x          -- apply f zero times
1 = λf. λx. f x        -- apply f once
2 = λf. λx. f (f x)    -- apply f twice
n = λf. λx. f^n(x)
```

The teaching point is not to develop the entire arithmetic of Church numerals, but to show that data can be represented by what it does.

This section should be taught as a conceptual payoff from untyped lambda calculus:

- if functions are the only primitive notion,
- then booleans and numbers must be encoded functionally,
- and surprisingly, this works.

---

3. Why Untyped Systems Need Discipline

Up to this point, students have seen the power of untyped computation:

- a tiny syntax,
- a universal model of computation,
- data represented by behavior,
- and two foundational models, Church and Turing.

The next question is:

> If untyped systems are so expressive, why introduce types at all?

The Standard Bad Example: Self-Application

The term

[= (x., x x)(x., x x)]

is the standard warning sign in untyped lambda calculus.

Why Does Ω Loop Forever?

Apply one beta-reduction step:

[(x., x x)(x., x x) ((x., x x)(x., x x))]

The result is exactly the same term we started with.

So after one reduction step, nothing has become simpler. If we reduce again, we get the same term again. Hence the reduction process repeats forever.

This is not a contradiction. It is a case of **non-termination**:

- the system keeps computing,
- but never reaches a normal form.

Why This Matters

This example is useful because it isolates the core problem:

- unrestricted self-application allows terms to feed themselves back into themselves,
- that feedback can destroy normalization,
- and it shows that bare computation needs discipline if we want a stable mathematical language.

Transition to Types

---

4. From Programming Types to Mathematical Types

Students already know the programming intuition:

```
int x = 42;       // OK
int y = "hello"; // ERROR
```

In Lean, the same basic discipline appears:

```
#check 42
#check "hello"
#check (42 + "hi")
```

The key transition is this:

- In programming, types prevent nonsense in computation.
- In type theory, types also organize mathematical meaning.

This is the exact moment when the lecture moves away from pure computation and toward foundations. The class should feel that we are no longer only asking how expressions run. We are asking which expressions count as meaningful mathematical objects.

Core Judgment

In type theory, the primitive form is

$[\, a : A \,]$

This should be read as:

- `a` is given as an object of type `A`.
- The expression is a **judgment**, not merely a proposition.

This differs from set-theoretic language:

[a A]

Set theory begins with membership.
Type theory begins with typed existence.

This distinction is philosophical as well as formal:

- In set theory, one imagines an object first, then asks which sets it belongs to.
- In type theory, objects are introduced already as inhabitants of a type.

This is the first major idea students must retain.

---

5. Judgment vs Proposition

The next distinction is technical and foundational.

In Set Theory

`a ∈ S` is a proposition.

It makes sense to ask whether it is true or false.

In Type Theory

`a : A` is a judgment.

For example, `π : Nat` is not a false theorem inside the system. It is ill-typed syntax and is rejected before mathematical reasoning begins.

Useful Contrast

- Proposition: something that may be true or false.
- Judgment: a well-formed assertion made by the system.

This is one reason type theory feels closer to syntax, construction, and computation.

Lean Anchor: Projections from a Proof of `A ∧ B`

```
import Mathlib

variable (A B : Prop)
variable (h : A ∧ B)

#check h.1
#check h.2
#check h.1 = h.2

example : B := h.right
example : A := h.left
```

```
def andSwap : B ∧ A := ⟨h.2, h.1⟩
#print andSwap
```

This is a useful early example because it already shows several ideas:

- a proof of `A ∧ B` contains data,
- conjunction behaves like a pair,
- extracting `.left` and `.right` is computational,
- rearranging the pair is already a proof.

---

6. Why Types Are Added: Simply Typed Lambda Calculus

Untyped lambda calculus is expressive, but too permissive.

For example:

[x, x]

or the self-application term:

[(x., x x)(x., x x)]

The point is not merely that these are strange; the point is that typing is introduced to classify terms and exclude nonsense.

This section should be presented explicitly as a response to the previous one:

- untyped lambda calculus explains pure computation,
- typed lambda calculus explains disciplined computation.

Why Add Types at All?

This is the core question of the middle part of Week 2.

Types are added because untyped computation has no built-in distinction between:

- meaningful application,
- accidental nonsense,
- and self-application patterns that create pathological behavior.

In other words, untyped lambda calculus gives a very powerful language of computation, but it does not by itself tell us which terms should count as well-behaved mathematical objects.

Three reasons are especially important.

1. Types separate meaningful expressions from nonsense

In an untyped setting, one can write terms such as

[x,x]

without any prior discipline.

But in typed settings, an application only makes sense if the input type matches the function's domain.

This is the first gain:

- not every syntactically legal expression counts as meaningful,
- type checking rejects malformed application.

2. Types control self-application

Some of the most dangerous terms in untyped lambda calculus come from unrestricted self-application.

The standard example is:

[(x., x x)(x., x x)]

which reduces to itself forever.

The pedagogical point here is not merely "infinite loops are bad." The deeper point is that unrestricted formal systems easily generate unstable or pathological terms unless extra discipline is imposed.

3. Types make the logic/computation bridge possible

Once types are introduced, terms can be classified by mathematical meaning.

That is what makes Curry–Howard possible:

- propositions can be treated as types,
- proofs can be treated as terms,
- and checking a proof becomes checking that a term has the right type.

So typed lambda calculus is not only a safer programming language. It is the doorway from pure computation to proof theory.

STLC

Add:

- base types such as `Bool`, `Nat`
- function types `A → B`

Typing rules:

- if `x : A` and `e : B`, then `λx. e : A → B`
- if `f : A → B` and `a : A`, then `f a : B`

Main Message

Typing does not just annotate terms. Typing constrains computation.

An important sentence for the lecture is:

> Typed lambda calculus is what turns bare symbolic computation into a disciplined mathematical language.

Students should also hear the first clean form of Curry–Howard:

- propositions correspond to types,
- proofs correspond to terms,
- implication corresponds to function type.

Lean Anchor: Function Application and Composition

```
variable {A B C : Type}
variable (F : A -> B)
variable (G : B -> C)
variable (a : A)

#check G (F a)
#check G ∘ F
```

This is a minimal Lean example for the operational content of implication:

- a function takes input of one type and returns output of another,
- composition corresponds to chaining implications.

Lean Anchor: Conditionals in a Typed Setting

Only after typed structure has entered the story is it worth showing the Lean version of conditionals:

```
#check ite
#check ite true
#check ite true 10 0
#eval ite false 10 20

def myIte (b : Bool) (t e : α) : α :=
  match b with
  | true => t
  | false => e

#eval myIte true 10 20
#eval myIte false 10 20
```

This is the right place for this code, because by now students have already seen:

- untyped computation,
- behavior-as-data,
- and the reason typed systems matter.

So Lean's conditional can now be understood as a typed version of branching, rather than as an isolated syntax trick.

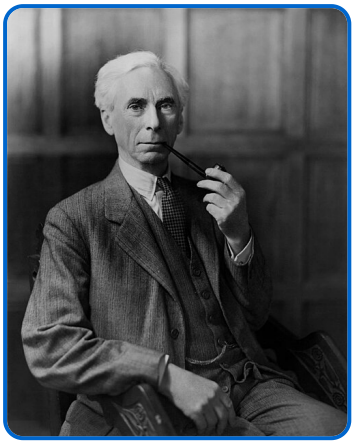
---

7. Russell's Paradox and Type-Theoretic Discipline

Now that types have been introduced, the lecture can return to a foundational payoff.

The benefit is not only that types make programs safer. The broader type-theoretic discipline also blocks some of the self-reference that generates foundational paradoxes.

Russell's Paradox



```
<strong>Bertrand Russell</strong> (1872–1970)
was one of the central figures in the
foundations of mathematics.
His paradox became one of the clearest
demonstrations that unrestricted self-
reference is untenable.
```

Naive set formation leads to self-reference:

[$R = \{x \; x \; x\}$]

Then we ask:

- If $R \in R$, then by definition $R \notin R$.
- If $R \notin R$, then by definition $R \in R$.

Contradiction.

What This Shows

Russell's paradox is not an infinite loop. It is a contradiction.

That is the contrast with the untyped lambda-calculus example Ω:

- Ω gives non-termination,
- Russell's paradox gives inconsistency.

But both come from dangerous forms of unrestricted self-reference.

Why Types Help

Type theory imposes discipline by stratification.

In Lean:

```
#check Nat
#check Type
#check Type 1
```

So we have a universe hierarchy:

- `Nat : Type`
- `Type : Type 1`
- `Type 1 : Type 2`

This blocks the kind of self-reference that creates Russell-style paradoxes.

At the handout level, it is worth stating the conceptual lesson plainly:

- untyped expressiveness is attractive,
- unrestricted formation principles are dangerous,
- and type-theoretic structure rules out some bad constructions automatically.

---

8. Curry–Howard: Proofs as Programs

This is the conceptual bridge from logic to type theory.

Useful correspondences:

- `A ∧ B` corresponds to a product/pair
- `A → B` corresponds to a function
- a proof of `A` corresponds to a term of type `A`

Example:

To prove `A ∧ B → B ∧ A`, write a function that swaps the pair.

```
def andSwap (h : A ∧ B) : B ∧ A :=
  ⟨h.2, h.1⟩
```

Students should see that:

- the function is the proof,
- type checking is proof checking.

Lean Anchor: Disjunction Constructors

```
variable (A B : Prop)

#check A ∨ B
```

```
example (a : A) : A ∨ B := Or.inl a
example (b : B) : A ∨ B := Or.inr b
```

This is a good place to contrast:

- conjunction as pair-like data,
- disjunction as a tagged choice.

---

9. Why Simple Types Are Not Enough

Ordinary function types have the form:

[A B]

where the output type `B` is fixed.

But mathematics often needs types that depend on values.

Motivating Example

Ordinary lists:

- `List Nat`

lose information about length.

What we want conceptually is something like:

- "lists of length exactly `n`"

This motivates indexed families such as vectors.

Lean Anchor: Indexed Families

```
#check Vector String
#check Vector String 0 = Vector String 1
```

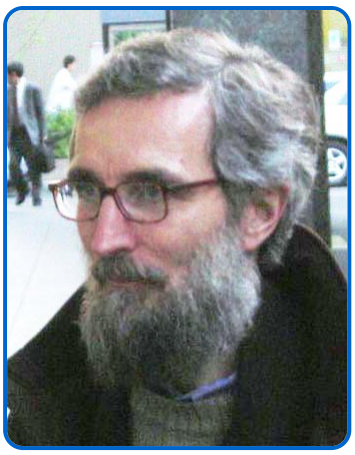
Students should notice immediately that:

- `Vector String 0` and `Vector String 1` are different types,
- length information lives at the type level.

---

10. Dependent Types

This is the central structural idea of the lecture.



```
<strong>Per Martin-Löf</strong> (b. 1942)
developed intuitionistic type theory in a form
that directly shaped modern dependent type
theory.
This is the main historical source behind the
style of reasoning used in Lean.
```

Type Families

A dependent type is a type that depends on a value.

Example:

[:]

So:

- `Vec Nat 0`
- `Vec Nat 1`
- `Vec Nat 3`

are different types.

Pi Types

[(x : A), B(x)]

This is the type of dependent functions.

Interpretation:

- constructive universal quantification,
- "for every `x : A`, produce something of type `B(x)`."

Sigma Types

[(x : A), B(x)]

This is the type of dependent pairs.

Interpretation:

- witness plus evidence,
- constructive existential quantification.

Students should leave this section knowing:

- `Π` corresponds to `forall`,
- `Σ` corresponds to `exists`,
- but both are computational objects, not only logical symbols.

Lean Anchor: `Π` and `∀`

```
def noneg := Π x : Nat, x ≥ 0
def noneg' := ∀ x : Nat, x ≥ 0

example : noneg = noneg' := rfl
```

In Lean:

- `Π x : A, B x` is the general dependent form,
- `∀ x : A, P x` is notation used when the codomain is a proposition.

Lean Anchor: `∃`

```
def geqfive := ∃ x : Nat, x ≥ 5

lemma geqfive' : ∃ x : Nat, 5 ≤ x :=
  ⟨5, Nat.le_refl 5⟩

#print geqfive'
```

This example should be explained as:

- an existential proof contains a witness,
- together with evidence about that witness.

Lean Anchor: An Ordinary Mathematical Predicate

One important point for students is that Lean does not only talk about toy examples. It can express ordinary mathematical predicates directly:

```
def prime := fun p =>
  p > 1 ∧
  ∀ a b : ℕ, a > 0 -> b > 0 -> a * b = p -> (a = p ∨ b = p)

#check prime
```

This is a good example to place in the main text because it shows that once propositions are treated as types, familiar mathematical definitions can be written inside the same language.

Lean Anchor: A Type Family Depending on a Value

```
abbrev dt := fun x : Nat => if x = 0 then Nat else String

#check dt 0
```

This is a compact illustration that a type can genuinely depend on a value.

---

11. Inductive Types and Mathematical Construction

Dependent type theory does not only talk about functions. It also builds data by constructors.

Natural Numbers

```
inductive Nat where
  | zero : Nat
  | succ : Nat → Nat
```

This encodes the idea that every natural number is built from:

- zero
- successor

Lean Anchor: Constructors Compute

```
example : Nat.succ 0 = 1 := rfl
example : Nat.succ 1 = 2 := rfl
example : Nat.succ (Nat.succ 0) = 2 := rfl
```

Induction

Induction is built into the elimination principle of the type.

This should be explained as:

- not an external theorem schema,
- but a consequence of how the type is constructed.

Lean Example

```
theorem add_zero : ∀ n : Nat, n + 0 = n := by
  intro n
  induction n with
  | zero => rfl
  | succ n ih => simp [Nat.succ_add, ih]
```

This is a useful anchor theorem for the lecture.

Lean Anchor: Lists Are Constructor Expressions

```
example : [1] = List.cons 1 .nil := rfl
example : [1, 2] = List.cons 1 [2] := rfl
```

```
example : [1, 2] = List.cons 1 (List.cons 2 .nil) := rfl

def onetoten := [1,2,3,4,5,6,7,8,9,10]
#print onetoten
```

This is useful for emphasizing that notation is not magic:

- lists are built from constructors,
- notation expands to ordinary inductive terms.

Historical Note: From Martin-Löf to Calculi of Constructions

This is a good transition point in the lecture:

- Martin-Löf provides the dependent-type viewpoint,
- later systems combine dependent types, lambda calculus, and richer universe mechanisms,
- this line eventually leads to the formal systems underlying Coq and Lean.

---

12. Preview of Week 3

Week 3 will pick up the next major topic:

- identity types,
- equality as paths,
- path induction and transport,
- and the first genuine entry point into HoTT.

For Week 2, it is enough to stop after dependent and inductive structure. That keeps the current lecture focused and avoids rushing the equality material.

---

13. Recommended Visual Assets

The handout suggests these figures for the slide stage:

1. Programming types: simple C/Lean comparison.
2. Russell paradox: Russell portrait plus contradiction diagram.
3. Universe hierarchy: `Type`, `Type 1`, `Type 2`.

---

14. Suggested Lean Thread for the Lecture

To keep the code examples coherent in class, the Lean snippets can be presented in this order:

1. `h.left`, `h.right`, `andSwap`

2. `f`, `g`, `F`, and bound versus free variables
3. `CBool`, `ctrue`, `cfalse`, `cIte`
4. `G (F a)` and `G ∘ F`
5. `ite` and `myIte`
6. `Vector String 0` versus `Vector String 1`
7. `noneg` and `noneg'`
8. `geqfive'`
9. `Nat.succ` examples
10. list constructor expansions

---

15. From Handout to Slides

When converting this handout to slides, the following rule should hold:

- one teaching point per slide,
- one example per slide,
- one main visual idea per slide.

Avoid combining:

- definition,
- comparison,
- code,
- and takeaway

on the same slide unless the page remains visually sparse.

The slide deck should be derived from the handout in this order:

1. Fix the lecture narrative.
2. Choose which examples are live-coded.
3. Choose which concepts get a diagram.
4. Split each subsection into small slide-sized units.
5. Export PNG and inspect page by page.

---

16. Open Questions for Discussion

Before final slide design, these points should be agreed:

1. How much HoTT should appear in Week 2, versus being delayed to Week 3?
2. Should `Church encodings` stay as a conceptual interlude, or be taught as a full subsection?
3. How much Lean code should be shown live versus placed in links/appendix?

4. Is the primary pacing goal conceptual clarity, or giving students enough Lean material to practice immediately after class?